DermAI

Skin Disease Detection System

An End-to-End MLOps Application

Course	DA5402 — Machine Learning Operations
Institute	Indian Institute of Technology Madras
Author	Harshit (`da25s003`)
Github	github.com/harshit-8118/E2E_Project_DA5402
MLFLOW	dagshub.com/da25s003/E2E_Project_DA5402.mlflow
DagsHub	dagshub.com/da25s003/E2E_Project_DA5402
	`It may required your own account`
Dataset	HAM10000 — 10,015 dermoscopy images, 7 classes
Model	EfficientNetB3 | Macro F1: 0.7423
Date	April 2026

Accompanying Documentation

The following detailed documents are provided in `project_reports/` and should be read alongside this report:

- **HLD_DermAI.pdf** — Architecture diagram and design rationale
- **LLD_DermAI.pdf** — All API endpoint I/O specifications
- **TestPlan_DermAI.pdf** — Test cases, acceptance criteria, and results
- **Grafana_dashboard.pdf** — System and Application Dashboard.
- **UserManual_DermAI.pdf** — Non-technical step-by-step usage guide

This application is for academic research purposes only and is not a certified medical device. Results must not be used as the sole basis for any clinical decision.

Contents

1 Problem Definition and Motivation **3**
1.1 Business Problem . . . 3
1.2 Measurable Success Metrics . . . 3
1.3 Disease Classes . . . 3

2 Data Collection, Versioning, and Preprocessing **3**
2.1 Dataset . . . 4
2.2 Data Versioning with DVC . . . 4
2.3 DVC Pipeline — Prepare Stage . . . 4
2.4 Baseline Statistics for Drift Detection . . . 4

3 Model Development and Training **5**
3.1 Model Architecture . . . 5
3.2 Training Configuration . . . 5
3.3 Class Imbalance Handling . . . 5
3.4 Evaluation Results . . . 6
3.5 MLflow Experiment Tracking . . . 6
3.6 CPU Deployment Challenge . . . 7

4 System Architecture and Software Design **7**
4.1 Architecture Overview . . . 7

5 Directory Structure **8**
5.1 API Design . . . 10
5.2 Key Design Decisions . . . 10

6 Frontend: UI/UX Design **11**
6.1 Authentication Flow . . . 11
6.2 User manual and Terms and Conditions . . . 11
6.3 Prediction Interface . . . 12
6.4 Design Principles . . . 13

7 Data Engineering — Airflow Pipeline **14**
7.1 DAG Overview . . . 14
7.2 Drift Detection . . . 14

8 Containerisation and Deployment **15**
8.1 Docker Services . . . 15
8.2 Healthcheck Dependency Chain . . . 16
8.3 Dockerfile Optimisation . . . 16
8.4 Data Persistence . . . 17
8.5 MLflow Model Serving — `serve.sh` . . . 17

9 Monitoring, Alerting, and Observability **17**
9.1 Prometheus Instrumentation . . . 17
9.2 Alert Rules . . . 18

9.3 Grafana Dashboard . . . 19
9.4 Alertmanager Email Notifications . . . 19

10 Source Control and Continuous Integration **20**
10.1 Git History and Branch Strategy . . . 20
10.2 DVC CI Pipeline . . . 21
10.3 MLproject Reproducibility . . . 21

11 Security Implementation **22**

12 Testing **23**
12.1 Test Infrastructure . . . 23
12.2 Results Summary . . . 23

13 Challenges and Resolutions **24**

14 Reproduction Guide **26**
14.1 Prerequisites . . . 26
14.2 Environment Setup . . . 26
14.3 Running the Full Stack . . . 26
14.4 Service URLs . . . 27
14.5 Reproducing Training . . . 27
14.6 Running Tests . . . 27
14.7 Key `.env` Variables . . . 28

15 Summary and Evaluation Coverage **28**

1 Problem Definition and Motivation

1.1 Business Problem

Skin cancer is among the most prevalent cancers globally, yet early detection dramatically improves patient outcomes. Dermoscopy — magnified imaging of skin lesions — is the clinical standard for initial triage, but requires specialist expertise. DermAI provides an AI-assisted classification tool that analyses dermoscopy images and returns a ranked diagnosis with Grad-CAM explainability, enabling faster initial screening.

1.2 Measurable Success Metrics

Metric	Target	Achieved
Model Macro F1 (test set)	≥ 0.70	**0.7423**
API p95 inference latency (CPU)	$< 4\,\mathrm{s}$	$\times$
Auth flow (OTP verified signup)	Implemented	✓
System uptime (Docker restart policy)	`unless-stopped`	✓
Prometheus custom metrics	≥ 15	25+
Grafana dashboard panels	≥ 30	Implemented
AIRFLOW Drift detection	≥ 5	7 *stages* Alert on drift
Test pass rate(rigrous)	$\geq 90\%$	99 % (72/73)

Table 1: Project success metrics — targets and outcomes

1.3 Disease Classes

The HAM10000 dataset covers seven diagnostic categories:

Code	Disease	Risk Level
`mel`	Melanoma (skin cancer)	**HIGH**
`bcc`	Basal Cell Carcinoma	**MEDIUM**
`akiec`	Actinic Keratosis	**MEDIUM**
`nv`	Melanocytic Nevi (moles)	**LOW**
`bkl`	Benign Keratosis	**LOW**
`df`	Dermatofibroma	**LOW**
`vasc`	Vascular Lesion	**LOW**

Table 2: HAM10000 disease classification scheme

2 Data Collection, Versioning, and Preprocessing

2.1 Dataset

The **HAM10000** (Human Against Machine with 10,015 Training Images) dataset HAM10000 is the standard dermoscopy benchmark, hosted on Harvard Dataverse (`https://doi.org/10.1038/sdata.2018.161`) and available via Kaggle. It contains 10,015 multi-source dermoscopic images with ground-truth labels derived from histopathology, confocal microscopy, or expert consensus.

Key dataset characteristics:

- **Total images**: 10,015 across two image folders (`part_1`, `part_2`)
- **Test set**: ISIC 2018 Task 3 holdout (1,511 images, separate ground-truth CSV)
- **Severe class imbalance**: `nv` (67%) vs `df` (1.1%)
- **Image size**: Variable; resized to 336×336 during preprocessing

2.2 Data Versioning with DVC

All raw data, processed splits, and trained model weights are tracked via **DVC** (Data Version Control) with DagsHub as the remote storage backend:

Listing 1: DVC remote configuration

```
dvc remote add origin https://dagshub.com/da25s003/
   E2E_Project_DA5402.dvc
dvc push   # upload to DagsHub
dvc pull   # restore on a new machine
```

This ensures full reproducibility: any collaborator can restore the exact dataset version corresponding to a Git commit with `dvc pull`.

2.3 DVC Pipeline — Prepare Stage

The `prepare` stage (`src/data_proc/prepare.py`) implements:

1. **Lesion-level stratified split**: 85% train / 15% validation, splitting by `lesion_id` to prevent data leakage across augmented duplicates
2. **ISIC test holdout**: kept completely separate (never seen during training)
3. **Baseline statistics**: class distribution, mean/std per channel, age distribution — serialised to `data/reports/baseline_stats.json` for later drift detection
4. **Integrity checks**: missing file detection, schema validation against the metadata CSV

Split sizes: **8,518 train** / **1,497 val** / **1,511 test (holdout)**.

2.4 Baseline Statistics for Drift Detection

During EDA, the following statistics are recorded in `baseline_stats.json`:

- Per-class absolute and relative counts (used for PSI calculation)

- Patient age mean and variance
- Image count per acquisition modality

The Airflow DAG later computes **Population Stability Index (PSI)** per class against this baseline to detect distribution drift.

3 Model Development and Training

3.1 Model Architecture

EfficientNetB3 was selected after comparative experiments with `convnext_small` and `resnet_50`. EfficientNetB3 balances parameter efficiency and accuracy for medical imaging tasks, offering:

- 10.7M trainable parameters
- Compound scaling of depth, width, and resolution
- Strong transfer learning from ImageNet pre-training

The classifier head was replaced with a fully connected layer mapping to 7 output classes.

3.2 Training Configuration

Training was performed on an NVIDIA GPU (Linux lab machine) using the following hyperparameters, all managed through `params.yaml` for DVC reproducibility:

Hyperparameter	**Value**
Epochs	30 (early stopping patience: 15)
Learning rate	1×10^{-4}
Batch size	16
Image size	336×336
Scheduler	CosineAnnealingLR
Loss function	CrossEntropyLoss (class-weighted)
Mixup alpha	0.1
Label smoothing	0.0
Weight decay	0.0
AMP (mixed precision)	Enabled (CUDA)
Random seed	42

Table 3: Training hyperparameters (`params.yaml`) CPU machine

3.3 Class Imbalance Handling

The dataset has severe class imbalance (melanocytic nevi dominate at ∼67%). Two strategies were applied:

1. **Weighted cross-entropy loss**: inverse-frequency class weights computed from training split
2. **Mixup augmentation** ($\alpha = 0.1$): linear interpolation of training samples to improve generalisation on minority classes

3.4 Evaluation Results

Metric	Value
Test Macro F1	**0.7423**
Test Accuracy	Logged in MLflow
Acceptance threshold	0.70
Model status	`skin-disease-classifier@production`

Table 4: Final model evaluation on ISIC holdout test set

The macro F1 of **0.7423** exceeds the acceptance threshold and is consistent with published baselines on HAM10000 for non-ensemble single-model approaches.

3.5 MLflow Experiment Tracking

All experiments were tracked on **DagsHub MLflow** E2E_Project_DA5402.mlflow Beyond `autolog`, the following were explicitly logged:

- **Parameters**: all hyperparameters from `params.yaml`
- **Metrics per epoch**: train loss, val loss, macro F1, per-class F1, AUC etc.
- **Artifacts**: confusion matrix PNG, Grad-CAM sample grid, classification report TXT, `train_history.json`
- **Model**: registered as `skin-disease-classifier` with `@production` alias
- **Tags**: Git commit hash, run environment (CUDA/CPU), training date, manual or dvc run etc.

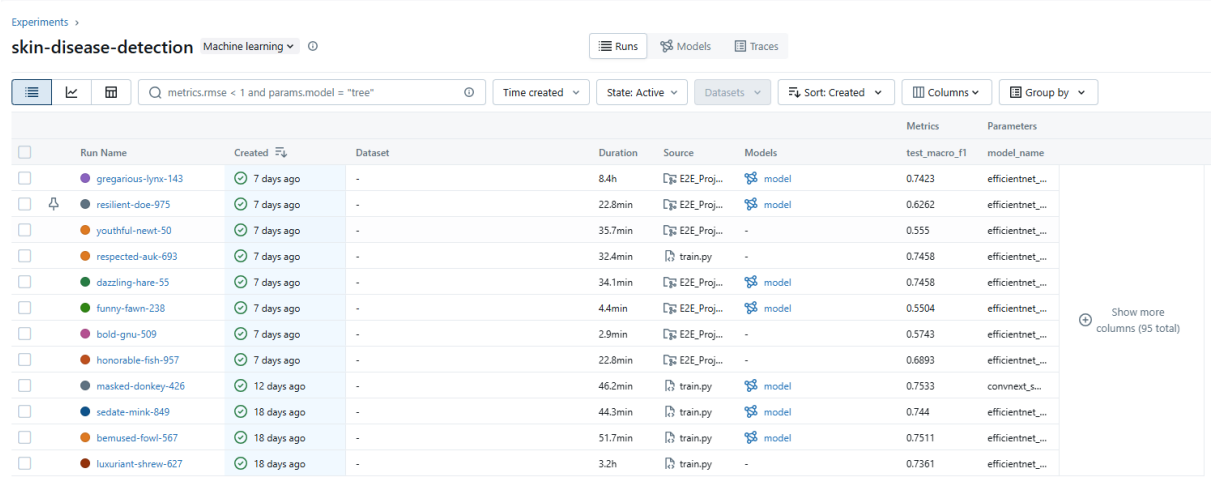


Figure 1: DagsHub MLflow showing multiple runs across EfficientNetB3 and ConvNeXt_small experiments, with macro F1 comparison chart

3.6 CPU Deployment Challenge

Challenge: CUDA Model Serving on CPU Machine

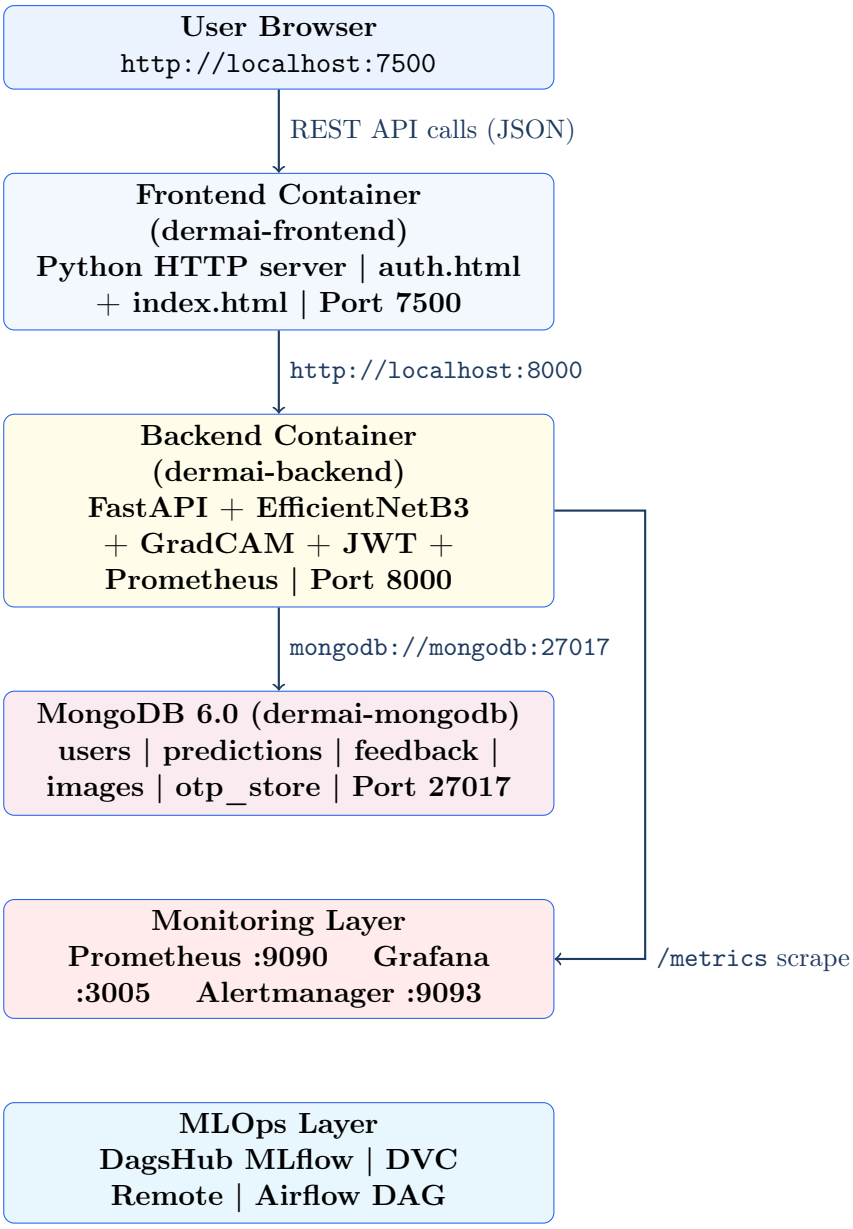
Training was performed on a Linux lab machine with CUDA. The MLflow registry stored the model as a Python pickle bound to `torch==2.11.0+cu130` and Python 3.10.12. Loading via `mlflow.pytorch.load_model()` on Windows CPU raised: `TypeError: code() argument 13 must be str, not int` — a Python bytecode version incompatibility.

Resolution: Exported clean CPU state-dict weights from the GPU machine using `torch.save(model.state_dict())` with `map_location="cpu"`, bypassing MLflow's pickle serialisation entirely. The backend loads weights directly: `torch.load(path, map_location="cpu", weights_only=True)`.

4 System Architecture and Software Design

4.1 Architecture Overview

DermAI follows a **loosely-coupled microservice architecture**. The browser communicates exclusively with the backend via REST API calls — the frontend has no direct database access and no shared code with the backend.



5 Directory Structure

```
E2E_Project_DA5402/
├── dvc.yaml ..................DVC pipeline stages: prepare → train → evaluate
├── params.yaml ..................................Centralized hyperparameters
├── MLproject ....................................................MLflow entry point
├── python_env.yaml ..................................Environment specification
├── docker-compose.yml ..............................10-service orchestration
├── docker/ ......................................Dockerfiles and service configs
│   └── mongod.conf
├── dags/ ..............................................Apache Airflow DAGs
└── monitoring/ ..................................Prometheus & Grafana stack
    └── prometheus.yml
```

```
│
├── alertmanager.yml.template ........................... replace credentials
├── rules.yml/
└── grafana/dashboards/
├── src/ ............................................... Core application source
│   ├── api/ ........................................... FastAPI backend
│   │   ├── main.py
│   │   ├── predict.py
│   │   ├── metrics.py
│   │   ├── serve_frontend.py
│   │   ├── deps.py
│   │   ├── auth.py
│   │   └── frontend/ ............................................Static web files
│   │       ├── *.html ....................................auth.html/index.html
│   │       ├── *.css ....................................style.css/login_style.css
│   │       └── *.js .....................................config.js/auth.js/app.js
│   ├── db/ ....................................................Database schemas
│   │   ├── mongodb.py
│   │   └── models.py
│   ├── models/ ................................................ML logic
│   │   ├── train.py
│   │   ├── inference.py
│   │   ├── model.py
│   │   └── aug_methods.py
│   ├── data_proc/ ........................................... Data preparation
│   │   ├── prepare.py
│   │   └── verify_files.py
│   ├── utils/ ................................................ Shared helpers
│   │   ├── logger.py
│   │   ├── mlflow_utils.py
│   │   ├── serve_weights.py
│   │   └── other utils ........................reproducibility/config/email_otp
│   ├── test/ ........................................... Testing & synthetic data
│   │   └── run_tests.py
├── outputs/models/ ..................................... DVC tracked artifacts
│   └── best_model_cpu.pth
├── data/ ...................................................DVC tracked datasets
│   ├── raw/
│   ├── processed/
│   ├── reports/ ....................................... GIT tracked small files
├── scripts/ ...................................................Shell utilities
│   ├── backup_mongodb.sh
│   ├── restore_mongodb.sh
│   ├── check_port.sh
│   └── serve.sh
└── Project_reports
    ├── E2E_REPORT.pdf
    ├── Grafana_dashboard.pdf
    └── HLD_DermAI.pdf
```

```
│
├── LLD_DermAI.pdf
├── TestPlan_DermAI.pdf
├── UserManual_DermAI.pdf
└── Screenshots of HLD, DAG, DVC
```

5.1 API Design

The backend exposes 16 REST endpoints across four groups:

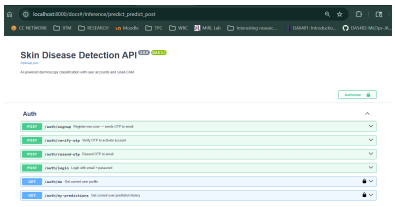
Group	Endpoint	Auth
System	`GET /health`, `/ready`, `/system/info`, `/classes`, `/metrics`	Public
Auth	`POST /auth/signup`, `/verify-otp`, `/resend-otp`, `/login`	Public
Auth	`GET /auth/me`, `/auth/my-predictions`	JWT
Inference	`POST /predict`, `/explain`, `/feedback`	JWT
Inference	`GET /feedback/stats`, `/predictions/stats`	JWT

Table 5: API endpoint summary (full I/O specifications in `LLD_DermAI.pdf`)

5.2 Key Design Decisions

Decision	Rationale
State-dict weights, not MLflow pickle	Avoids Python/CUDA version incompatibility between training and serving environments
`localhost` API URL in frontend	Browser JS runs on the host machine, not inside Docker; Docker service names are unresolvable from the browser
Lazy MongoDB reconnect	Backend starts before MongoDB is fully ready; `_ensure_connected()` retries every 5 s automatically
Background metrics thread	Prometheus scrapes every 10–15 s; middleware-only metrics go stale between requests; background thread updates CPU/memory every 5 s
`run_in_executor` for inference	`torch.no_grad()` blocks the async event loop; offloading to a thread pool keeps the API responsive under concurrent load

Table 6: Key design decisions and rationale



(a) Swagger authentication endpoint

(b) Available API routes

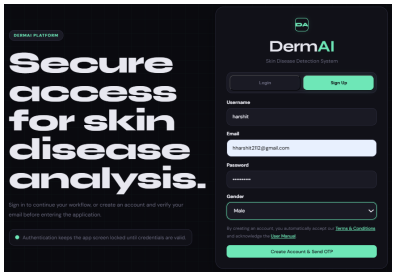
(c) Prediction endpoint

Figure 2: Swagger UI documentation for authentication, API listing, and prediction endpoints.

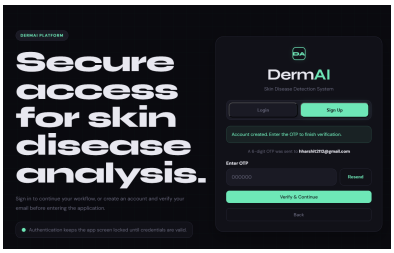
6 Frontend: UI/UX Design

The frontend is a single-page HTML/JS/CSS application served by `serve_frontend.py` (Python standard library HTTP server). It is intentionally dependency-free — no React, no npm build step — ensuring it can be rebuilt and deployed without a node toolchain.

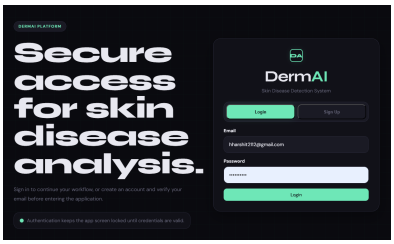
6.1 Authentication Flow



(a) User signup page



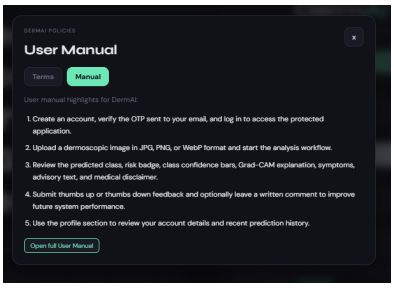
(b) OTP verification page



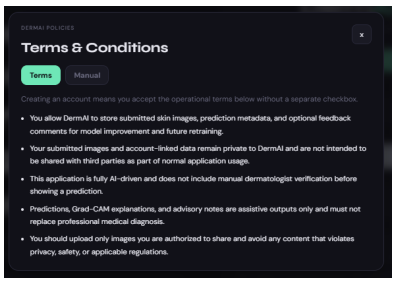
(c) User login page

Figure 3: Frontend authentication workflow showing signup, OTP verification, and login interfaces.

6.2 User manual and Terms and Conditions



(a) User Manual



(b) Terms and Conditions

Figure 4: Auto approved terms and conditions and User manual

The auth flow:

1. User registers with username, email, password, gender
2. Backend generates a 6-digit OTP, sends via Gmail SMTP
3. User enters OTP on the verification screen (TTL: 10 minutes)
4. Account is activated; user logs in and receives a JWT token
5. All subsequent requests carry `Authorization: Bearer <token>`

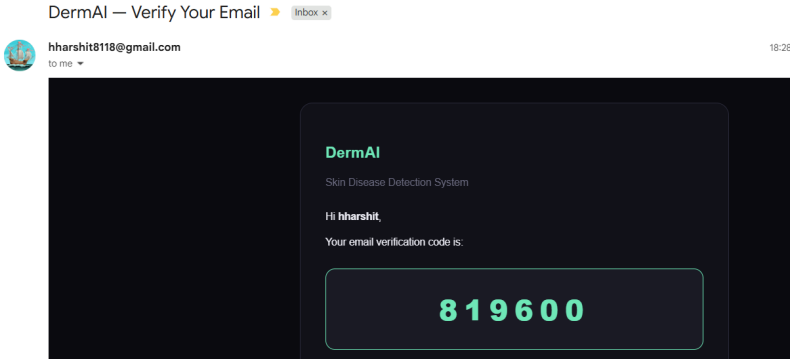


Figure 5: Gmail inbox showing the DermAI OTP email with 6-digit code, styled HTML email template

6.3 Prediction Interface

The prediction result panel displays:

- Predicted disease class with risk level badge (colour-coded)
- Confidence score and probability bars for all 7 classes
- Symptoms list and clinical advisory text
- **Grad-CAM heatmap**: jet-coloured overlay showing which image regions drove the model's decision
- Medical disclaimer
- Thumbs up / down feedback buttons

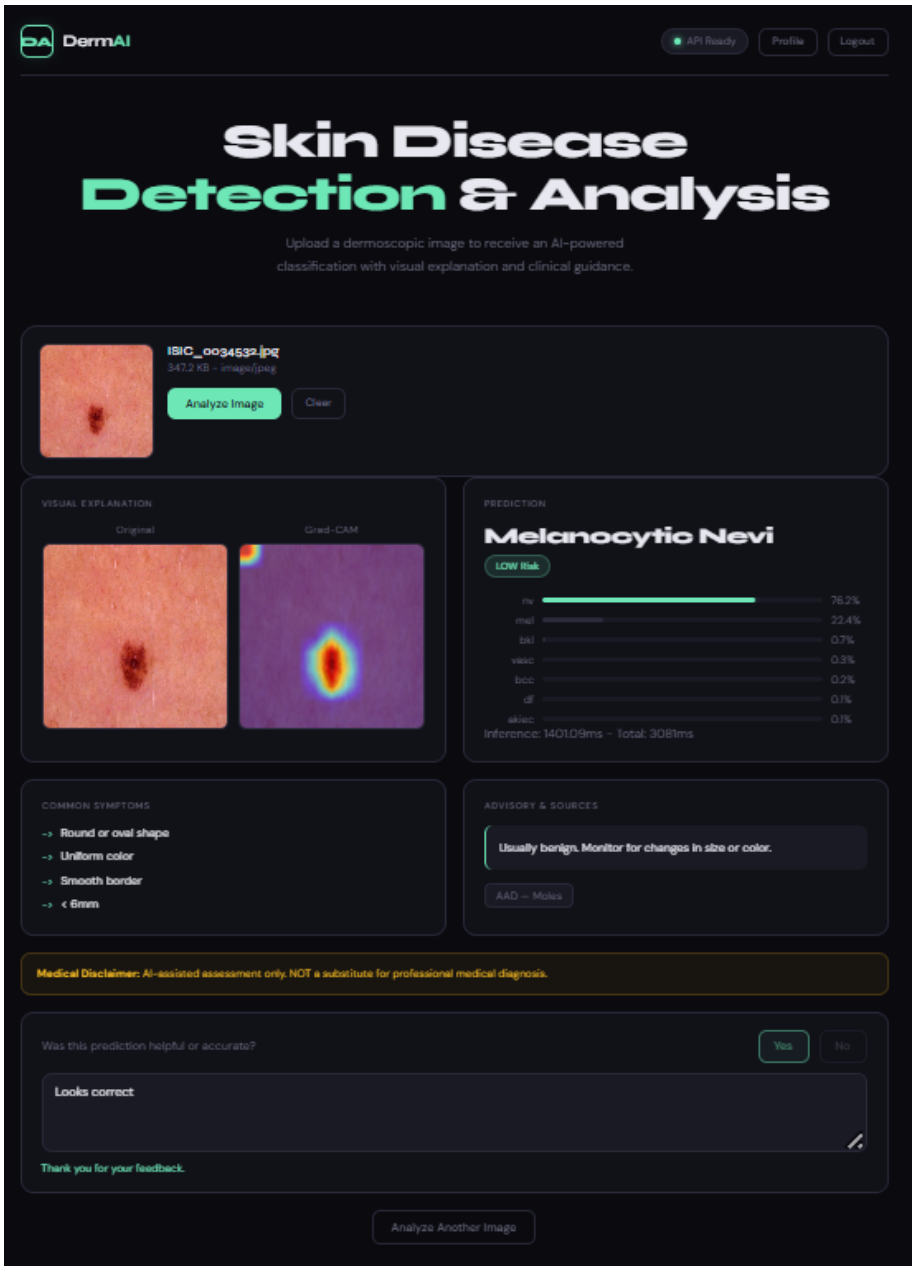


Figure 6: Drag-and-drop upload zone; result panel showing predicted class, risk badge (HIGH/MEDIUM/LOW), confidence score, all-class scores, symptoms, advisory text, and Grad-CAM heatmap overlay

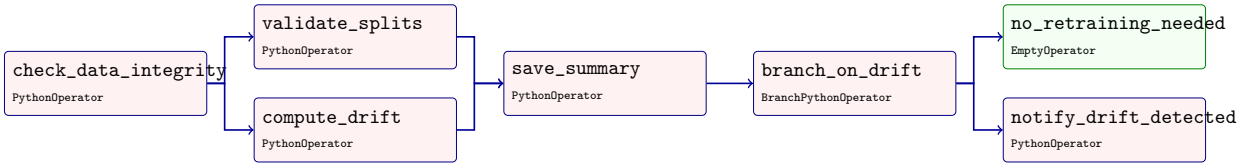
6.4 Design Principles

- **Dark medical aesthetic**: dark backgrounds (`#0a0a0f`) with teal accent (`#6ee7b7`) convey clinical professionalism
- **Loose coupling**: `config.js` is generated at container startup with the runtime API URL; the frontend contains no hardcoded backend addresses
- **Accessibility**: form validation, OTP resend timer, graceful error messages

7 Data Engineering — Airflow Pipeline

7.1 DAG Overview

A nightly Airflow DAG (`dermai_data_ingestion`, schedule: `0 2 * * *`) validates data health and detects distributional drift:



7.2 Drift Detection

The `compute_drift` task monitors **metadata distribution drift** using the class distribution stored in the training baseline report. Since the baseline does not include patient-age statistics, **age drift is not used** in the final pipeline.

Let p_i^{baseline} and p_i^{current} denote the baseline and current proportions for class i, over the seven disease classes. The DAG computes the following drift signals:

1. Population Stability Index (PSI):

$$\text{PSI}_i = \left(p_i^{\text{current}} - p_i^{\text{baseline}}\right) \ln\left(\frac{p_i^{\text{current}}}{p_i^{\text{baseline}}}\right)$$

and the aggregate score

$$\text{PSI}_{\text{total}} = \sum_{i=1}^{7} |\text{PSI}_i|$$

2. Per-class relative shift:

$$\Delta_i = \frac{\left|p_i^{\text{current}} - p_i^{\text{baseline}}\right|}{\max\left(p_i^{\text{baseline}}, \epsilon\right)}$$

where ϵ is a small constant for numerical stability.

3. Jensen–Shannon divergence (JSD):

$$\text{JSD}(P \parallel Q) = \frac{1}{2} D_{\text{KL}}(P \parallel M) + \frac{1}{2} D_{\text{KL}}(Q \parallel M), \quad M = \frac{1}{2}(P + Q)$$

4. Total variation distance (TVD):

$$\text{TVD}(P, Q) = \frac{1}{2} \sum_{i=1}^{7} \left|p_i^{\text{current}} - p_i^{\text{baseline}}\right|$$

5. Sample volume change:

$$\text{SampleRatioChange} = \frac{|N_{\text{current}} - N_{\text{baseline}}|}{\max(N_{\text{baseline}}, 1)}$$

A drift alert is triggered if any of the following conditions hold:

- any class has relative shift $> 5\%$,
- total PSI > 0.1,
- Jensen–Shannon divergence > 0.02,
- total variation distance > 0.12,
- sample volume change $> 20\%$,
- any expected class is missing, or any unseen class appears.

Rather than triggering `dvc repro` automatically, which is impractical on a CPU-only demo machine, the DAG routes to an **alert email task**. This task generates a retraining request report and notifies the operator by email, so that retraining can be reviewed and executed later on an appropriate GPU-capable environment.

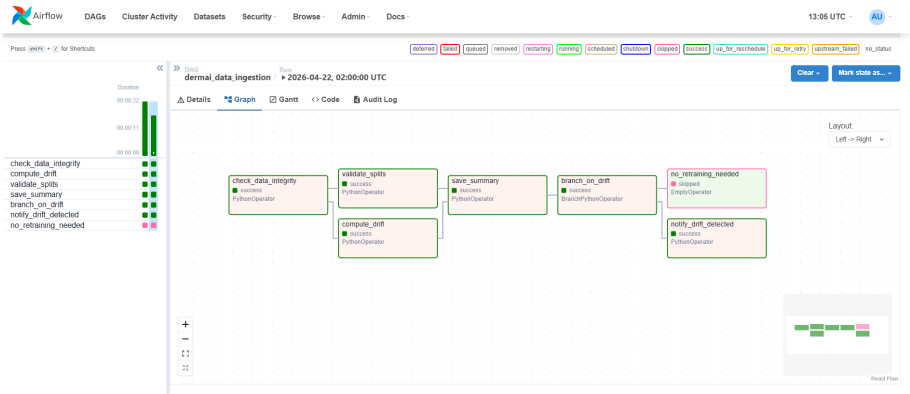


Figure 7: DAG graph showing all 6 tasks with colour-coded status (green = success), branching logic visible at `branch_on_drift`

8 Containerisation and Deployment

8.1 Docker Services

The complete stack is defined in a single `docker-compose.yml`:

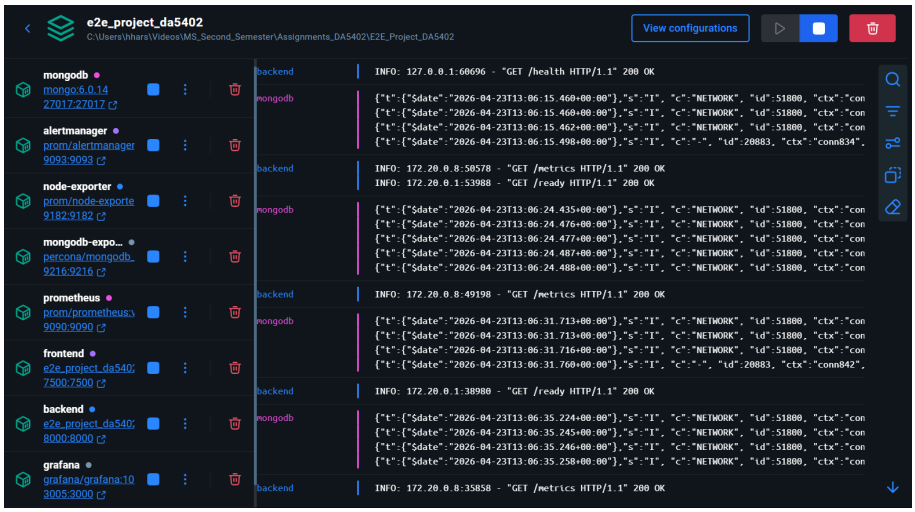


Figure 8: `docker compose ps` or Docker Desktop UI showing all 10 containers in `running` / `healthy` state

Container	Image	Port
`dermai-mongodb`	`mongo:6.0.14`	27017
`dermai-backend`	Custom (Python 3.10-slim + uv)	8000
`dermai-frontend`	Custom (Python 3.10-slim)	7500
`dermai-prometheus`	`prom/prometheus:v2.51.2`	9090
`dermai-grafana`	`grafana/grafana:10.4.3`	3005
`dermai-alertmanager`	`prom/alertmanager:v0.27.0`	9093
`dermai-node-exporter`	`prom/node-exporter:v1.8.2`	9182
`dermai-mongodb-exporter`	`percona/mongodb_exporter:0.40.0`	9216
`dermai-airflow-web`	`apache/airflow:2.9.2-python3.10`	8080
`dermai-airflow-scheduler`	`apache/airflow:2.9.2-python3.10`	—

Table 7: Docker Compose service inventory

8.2 Healthcheck Dependency Chain

Services start in a defined order enforced by `depends_on` conditions:

$$\texttt{mongodb (healthy)} \rightarrow \texttt{backend (healthy)} \rightarrow \texttt{frontend}$$

$$\texttt{backend} \rightarrow \texttt{prometheus (healthy)} \rightarrow \texttt{grafana}$$

8.3 Dockerfile Optimisation

The backend Dockerfile uses **uv** (Astral) for fast dependency installation with `-mount=type=cache` layer caching, reducing rebuild time from ∼8 minutes to ∼90 seconds on unchanged requirements.

Listing 2: Layer-cache optimised installation

```
COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/
COPY docker-requirements.txt .
```

```
RUN --mount=type=cache,target=/root/.cache/uv \
    uv pip install --system -r docker-requirements.txt
```

8.4 Data Persistence

All stateful data persists in named Docker volumes that survive `docker compose down`:

- `mongodb_data` — all user, prediction, feedback, and image data
- `prometheus_data` — 15 days of time-series metrics
- `grafana_data` — dashboard configurations and user settings

Only `docker compose down -v` destroys volumes. Backup scripts (`backup_mongodb.sh` / `restore_mongodb.sh`) allow safe pre-maintenance snapshots.

8.5 MLflow Model Serving — `serve.sh`

The `serve.sh` script launches `mlflow models serve` against the registered production model, satisfying the MLflow APIification requirement:

Listing 3: serve.sh — MLflow model server startup

```
MODEL_URI="models:/skin-disease-classifier@production"
mlflow models serve -m "$MODEL_URI" --port 5001 \
    --host 0.0.0.0 --env-manager local
```

The FastAPI backend also supports loading from the registry (`USE_REMOTE_MODEL=true`) for environments where the GPU-trained pickle is compatible with the runtime.

9 Monitoring, Alerting, and Observability

9.1 Prometheus Instrumentation

All 20+ custom metrics are defined in `src/api/metrics.py` and updated via a background thread running every 5 seconds (independent of HTTP traffic):

Metric	Type	What it tracks
`api_requests_total`	Counter	Every HTTP request by endpoint/method/status
`predictions_total`	Counter	Per disease class and risk level
`inference_latency_seconds`	Histogram	End-to-end model inference time
`prediction_confidence`	Histogram	Model output confidence distribution
`feedback_total`	Counter	Thumbs up / down counts
`user_predictions_total`	Counter	Per-username prediction count
`user_rate_limit_breach_total`	Counter	Users exceeding request threshold
`system_cpu_percent`	Gauge	Host CPU utilisation (psutil)
`system_memory_percent`	Gauge	Host RAM utilisation (psutil)
`mongodb_up`	Gauge	MongoDB connectivity (1/0)
`model_loaded`	Gauge	Model readiness (1/0)
`users_registered_total`	Gauge	Total registered users
`feedback_positive_rate`	Gauge	Rolling thumbs_up / total ratio
`unique_predictions_total`	Gauge	Monotonic prediction counter
`errors_total`	Counter	Errors by type and endpoint
...	...	...

Table 8: Prometheus custom metrics — key subset

Background Metrics Thread

A critical fix was needed: `psutil.cpu_percent(interval=None)` returns 0.0 on its first call and is meaningless if called only on HTTP requests. A daemon thread calls `psutil` every 5 s, priming the internal timer on startup and ensuring Grafana always shows current CPU/memory values regardless of request volume.

9.2 Alert Rules

Fourteen alert rules are defined in `monitoring/rules.yml`, tuned for a 16 GB Intel i5 development machine to fire during load testing:

Alert	Condition	Severity
ModelNotLoaded	`model_loaded == 0` for 20 s	Critical
MongoDBDown	`mongodb_up == 0` for 45 sec	Critical
APIDown	`up{job="fastapi_backend"} == 0` for 45 sec	Critical
SystemCPU_Critical	CPU $>$ 70% for 45 s	Critical
SystemMemory_Critical	Memory $>$ 70% for 45 s	Critical
HighInferenceLatency	p95 latency $>$ 4 s for 45 sec	Critical
RequestMemory_Critical	p95 req memory $>$ 1000 MB for 45 s	Critical
UserRateLimitBreach	Any breach in 5 min window	Warning
NegativeFeedbackSpike	thumbs_down $>$ 1.5$\times$ thumbs_up for 1 min	Warning
RegistrationSpike	$>$ 10 new users in 5 min	Warning

Table 9: Alert rules summary

9.3 Grafana Dashboard

PDF is attached with the monitoring files, please refer to that...

- **Grafana Dashboard — System Status Row**
  - Model Status (green)
  - MongoDB Status (green)
  - API Uptime
  - Users Registered
  - Total Predictions
  - Feedback Rate
- **Grafana Dashboard — Inference Performance Row**
  - p50 / p95 latency time series
  - Request rate by endpoint
  - Active concurrent requests gauge
  - Error rate
- **Grafana Dashboard — User Activity Row**
  - Per-class prediction pie chart
  - Feedback distribution donut chart
  - Top-10 users bar chart
  - High-risk counter
- **Grafana Dashboard — System Resources Row**
  - CPU % time series with threshold lines
  - Memory % time series
  - Memory used gauge
  - Model size stat

9.4 Alertmanager Email Notifications

Alertmanager routes alerts via Gmail SMTP to three receivers: `critical_alerts`, `warning_alerts`, and `database_alerts`, each with a styled HTML email template.

(a) CPU alert (b) API down (c) High active requests (d) High inference latency (e) High request volume (f) High-risk predictions (g) MongoDB alert (h) Registration alert (i) Memory alert (j) User rate limit

Figure 9: Alertmanager email notifications for infrastructure health, API reliability, inference performance, user activity, and security thresholds.

10 Source Control and Continuous Integration

10.1 Git History and Branch Strategy

The project was developed incrementally across 20 commits, progressing from initial DVC setup through full dockerisation and monitoring:

Listing 4: Condensed Git log (most recent first)

```
f799305 (HEAD -> master, origin/master, origin/HEAD, dagshub/
   master) Dockerized changes | debugged airflow pipeline with
   new drift rules
c66fd7e Added test file easy moderate and rigorous | new profile
   features shows predictions | cleaned prepare dvc stage
a4a420a Added production rules
bd69027 torch inference was blocking parallelism, on backend is
   debugged | container recreated
95f8816 (tag: v1.0_train_effnet_b3) model training with
   efficientnet_b3 | 3 experiments created | model registry with
   version 4 | experiment name: sedate-mink-849 | clear dvc
   pipeline maintained | model version 1.0 | Test macro f1: 0.74
363b522 chore: untrack outputs from git, managed by dvc
beb0aef added dagshub.mlflow | reproducible metrics code
3c3bac1 trained efficientnet_b3 and convnext_small logged details
    into local mlflow, dagshub mlflow is setup
f0757f6 cleaned prepare dvc pipeline
e672bd0 pipeline: prepare stage - splits, baseline stats, dvc
   pipeline
d3b282b data: track raw data with dvc
8870940 dvc remote dagshub cred added"
a32a5c1 init: dvc setup
```

```
ab99764 Initial commit
```

10.2 DVC CI Pipeline

The three-stage DVC pipeline provides reproducible CI:

Listing 5: DVC pipeline stages

```
dvc dag
# prepare --> train --> evaluate

dvc repro           # run full pipeline
dvc repro prepare   # run single stage
dvc metrics show    # compare runs
dvc push            # sync artifacts to DagsHub
```

Each stage has explicit `deps` (inputs), `params` (from `params.yaml`), `outs` (cached outputs), and `metrics` (uncached JSON reports). Changing any dependency or parameter automatically invalidates downstream stages.

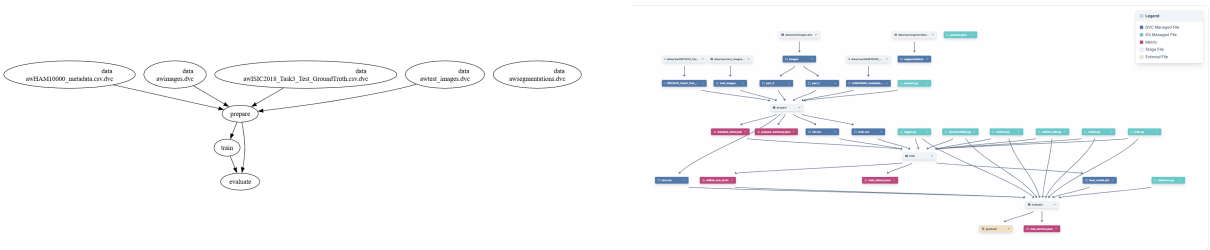


Figure 10: DVC pipeline visualizations: Left shows generated DAG image, right shows output of `dvc dag` with prepare → train → evaluate dependency chain.

10.3 MLproject Reproducibility

The `MLproject` file defines entry points for reproducible training:

Listing 6: MLproject entry points

```
entry_points:
  prepare:
    command: python -m src.data_proc.prepare
    --val_size {val_size}
        --random_seed {random_seed}
  train_and_evaluate:
    command:  python -m src.models.train
        --model_name {model_name}
        --epochs {epochs}
        --batch_size  {batch_size}
        --learning_rate {learning_rate}
        --image_size  {image_size}
        --random_seed  {random_seed}
        --use_weighted_loss {use_weighted_loss}
```

```
        --use_amp {use_amp}
        --weight_decay {weight_decay}
        --mixup_alpha {mixup_alpha}
        --cutmix_alpha {cutmix_alpha}
        --label_smoothing {label_smoothing}
        --early_stopping_patience {early_stopping_patience}
        --scheduler {scheduler} && python -m src.models.inference
        --model_path {model_path}
        --acceptance_macro_f1 {acceptance_macro_f1}
```

Any collaborator can reproduce training with:

```
mlflow run . --env-manager local -e train_and_evaluate
```

11 Security Implementation

- **Password storage**: SHA-256 hashed; plaintext never persisted
- **JWT authentication**: All inference endpoints require `Authorization: Bearer <token>`; tokens expire after 60 minutes
- **Email OTP**: 6-digit, 10-minute TTL (MongoDB TTL index on `otp_store`); prevents fake account creation
- **Secret management**: `.env` is gitignored; `.env.example` committed with placeholder values
- **Rate limiting**: Per-user request count tracked in `request_log` (TTL 1 hour); breach triggers Prometheus alert
- **CORS**: Configured for all origins in development; should be restricted to known domains in production

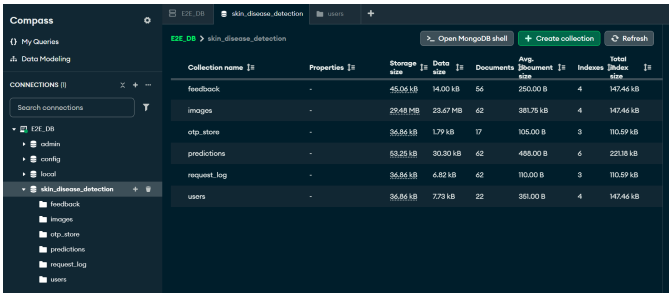


Figure 11: Compass connected to `mongodb://localhost:27017` showing 6 collections: users, predictions, feedback, images, otp_store, request_log with document counts

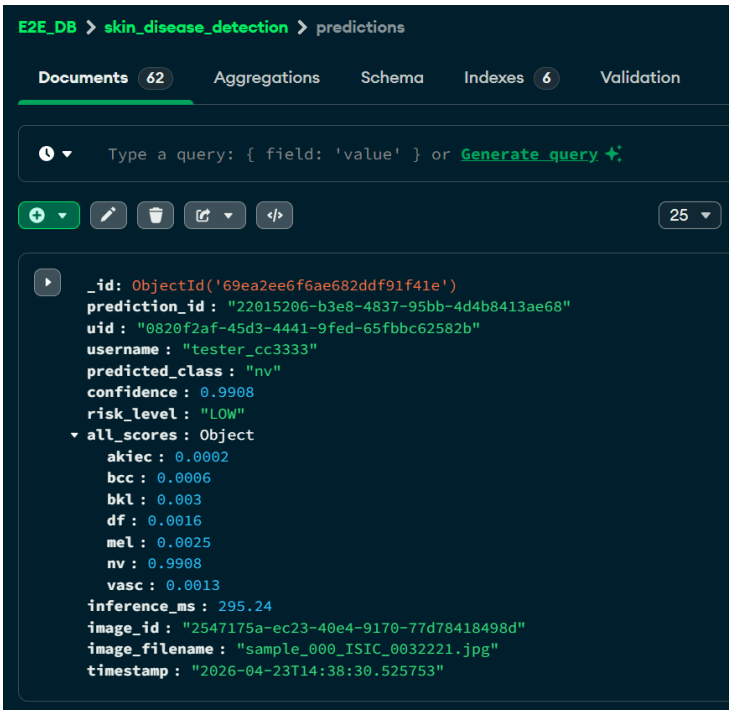


Figure 12: Sample prediction document showing uid, username, predicted_class, confidence, all_scores, image_id, risk_level, inference_ms, timestamp fields

12 Testing

12.1 Test Infrastructure

- **Real test images**: 100 ISIC dermoscopy images sampled via `src/test/data_creation.py` from `data/raw/test_images/`
- **Unified test runner**: `src/test/run_tests.py` with four modes: `easy`, `moderate`, `rigorous`, `overall`

12.2 Results Summary

Test Level	Cases	Passed	Pass Rate
Easy	14	14	100%
Moderate	22	22	100%
Rigorous	73	72	99%
UNION	**109**	**108**	$\approx 99\%$

Table 10: Test results summary

Notable rigorous test results:

- 25 concurrent predictions (12 users $\times$ 50 images) — 0 errors, p95 $\approx$ 7.2 s

- MongoDB data integrity verified: uid, username, image_id in every prediction record
- 14/14 Prometheus metrics present in registry
- 6/6 alert rules loaded and validated in Prometheus

Full test details are in `TestPlan_DermAI.pdf` in `project_reports/`.

13 Challenges and Resolutions

1. GPU Training Machine → CPU Serving Machine

Training was done on a Linux lab machine with CUDA GPU. The serving machine is a Windows laptop (Intel i5, no GPU). MLflow registry stored models as Python pickles bound to `+cu130` PyTorch and Python 3.10.12 bytecodes. Loading on a different Python version raised `TypeError: code() argument 13 must be str, not int`.
Resolution: Exported CPU-compatible weights using `torch.save(model.state_dict(), map_location="cpu")`. Backend now loads via `torch.load(..., weights_only=True)` bypassing pickle.

2. Intermittent MongoDB "Database Not Available" Error

The backend sometimes started before MongoDB was fully initialised. The original code called `mongo = MongoDB()` at module import time with a 3-second timeout — if MongoDB wasn't ready, `connected = False` was set permanently for the container lifetime.
Resolution: Rewrote `MongoDB` class with `_ensure_connected()` called on every operation. Retries every 5 s automatically. Docker healthcheck (`mongosh ping`) ensures backend only starts after MongoDB is truly ready.

3. Frontend "Failed to Fetch" / Swagger CORS Errors

Frontend Docker command initially used `-api-url http://backend:8000`. This resolves inside Docker's internal network but is unresolvable from the host browser (JavaScript runs on the user's machine, not inside Docker).
Resolution: Changed to `-api-url http://localhost:8000`. The `serve_frontend.py` writes `config.js` at startup with the correct host-accessible URL.

4. Prometheus Metrics Showing Zero Between Requests

`system_cpu_percent` and `system_memory_percent` were set only inside the HTTP middleware. Between requests, Prometheus scraped stale values. Also, `psutil.cpu_percent(interval=None)` returns 0.0 on its first call.
Resolution: Moved system metrics to a daemon thread updating every 5 s. Added a 1-second blocking warmup call at startup to prime the psutil internal timer.

5. Prometheus `inhibit_rules` Schema Error

Original `rules.yml` included `inhibit_rules` at the file root, causing Prometheus to fail on load: `field inhibit_rules not found in type rulefmt.RuleGroups`. **Resolution**: `inhibit_rules` belongs in `prometheus.yml` or `alertmanager.yml` only — it is not a valid key inside a rule file. Removed from `rules.yml`.

6. Grafana 401 WebSocket Token Rotation Errors

`grafana:12.1.1` changed session token behaviour, causing constant WebSocket 401 errors in the browser.
Resolution: Pinned to `grafana/grafana:10.4.3` (stable LTS) and added `GF_AUTH_TOKEN_ROTATION_INTERVAL_MINUTES=0`.

7. Parallelism: Active Requests Never Exceeded 1

`torch.no_grad()` and GradCAM run synchronously, blocking the entire asyncio event loop. Under concurrent load, every request queued behind the previous one; the `active_requests` gauge never exceeded 1.
Resolution: Offloaded both inference and GradCAM to the thread pool via `asyncio.get_event_loop().run_in_executor(None, fn)`. Added `_torch_lock` (threading.Lock) to serialise actual PyTorch calls safely. Event loop stays free; `active_requests` now shows 2–3 under load.

8. Slow Docker Build Times (∼8 minutes)

Every code change triggered a full pip install because requirements were not separated from source code in the Dockerfile layer order.
Resolution: Reordered Dockerfile layers (requirements first, source code last) and added `uv` with `-mount=type=cache`. Rebuild time dropped to ∼90 s for source-only changes.

9. DVC Repro vs `mlflow run .` Confusion

Initially unclear whether DVC or MLflow should be the CI entry point.
Resolution: DVC manages the **pipeline** (data → model → metrics, with caching and dependency tracking). MLflow manages **experiments** (what parameters were tried and what results they produced). `src/models/train.py` calls MLflow from inside the DVC stage — they are complementary, not alternatives.

10. Airflow SQLite "Unable to Open Database File"

Airflow init container failed with `sqlite3.OperationalError: unable to open database file`. The SQLite DB path (`/opt/airflow/airflow_db/airflow.db`) pointed to a Docker volume mount, but the directory did not exist inside the container before Airflow tried to create the file.
Resolution: Added `mkdir -p /opt/airflow/airflow_db` as the first command in the init container entrypoint, before `airflow db migrate`.

14 Reproduction Guide

14.1 Prerequisites

- Docker Desktop (Windows/Mac) or Docker Engine (Linux)
- Python 3.10+ (for local development and tests)
- DVC: `pip install dvc`
- Git and DagsHub access

14.2 Environment Setup

Listing 7: Environment setup

```
# 1. Clone repository
git clone https://dagshub.com/da25s003/E2E_Project_DA5402.git
cd E2E_Project_DA5402

# 2. Copy .env.example to .env and fill values
cp .env.example .env
# Required fields to set:
#   JWT_SECRET_KEY       python -c "import secrets; print(secrets
   .token_hex(32))"
#   GMAIL_APP_PASS       Gmail App Password (myaccount.google.com
    > Security)
#   AIRFLOW__CORE__FERNET_KEY     python -c "from cryptography.
   fernet import
#                                Fernet; print(Fernet.generate_key
   ().decode())"

# 3. Pull data and model weights from DagsHub
dvc pull
# This restores: data/raw/, data/processed/

python src/utils/serve_weigths.py
# This will store the weights from the production registory.
```

14.3 Running the Full Stack

Listing 8: Start all Docker services

```
# Create required host directories
mkdir -p logs backups dags

# Copy Airflow DAG
copy src\test\dermai_ingestion_dag.py dags\   # Windows
# cp src/test/dermai_ingestion_dag.py dags/   # Linux/Mac
```

```
# Start core services
docker compose --env-file .env up --build -d

# Verify all services healthy
docker compose ps
```

14.4 Service URLs

Service	URL	Credentials
Frontend	`http://localhost:7500`	Register new account
API Swagger	`http://localhost:8000/docs`	—
Prometheus	`http://localhost:9090`	—
Grafana	`http://localhost:3005`	admin / admin
Alertmanager	`http://localhost:9093`	—
Airflow	`http://localhost:8080`	admin / admin

Table 11: Service access URLs after `docker compose up`

14.5 Reproducing Training

Listing 9: Reproduce the DVC training pipeline

```
# Full pipeline (prepare + train + evaluate)
dvc repro

# Or via MLproject
mlflow run .--env-manager local -e train_and_evaluate

# Show metrics
dvc metrics show
mlflow ui --port 5001  # local MLflow server
```

14.6 Running Tests

Listing 10: Test execution

```
pip install requests pymongo Pillow numpy

# Sample 100 real ISIC images for testing
python src/test/data_creation.py

# Run test levels
python src/test/run_tests.py --mode easy
python src/test/run_tests.py --mode moderate
python src/test/run_tests.py --mode rigorous
python src/test/run_tests.py --mode overall   # + deliberate
   errors for alerts
```

14.7 Key `.env` Variables

Variable	Example	Note
`MODEL_PATH`	`outputs/models/best_model_cpu.pth`	CPU state-dict weights
`DEVICE`	`cpu`	cpu or cuda
`IMAGE_SIZE`	`336`	Must match training
`MONGO_URI`	`mongodb://mongodb:27017`	Docker service name
`JWT_SECRET_KEY`	32+ char random hex	Generate with `secrets`
`GMAIL_APP_PASS`	16-char Google App Password	OTP + Alertmanager SMTP
`GRAFANA_PORT`	`3005`	Avoids conflict with port 3000

Table 12: Key `.env` configuration variables

15 Summary and Evaluation Coverage

Evaluation Criterion	Max	Implemented
Demonstration		
Frontend UI/UX (intuitive, responsive, styled)	6	✓
User manual	—	✓(`UserManual_DermAI.pdf`)
ML Pipeline Visualization (Airflow + DVC + MLflow UIs)	4	✓
Software Engineering		
Architecture diagram (HLD)	2	✓(`HLD_DermAI.pdf`)
LLD with endpoint I/O specs	—	✓(`LLD_DermAI.pdf`)
Loose coupling (frontend ↔ backend via REST only)	—	✓
Logging, exception handling, PEP8	2	✓
Test plan + results	1	✓(`TestPlan_DermAI.pdf`)
MLOps Implementation		
Data engineering pipeline (Airflow DAG)	2	✓
DVC CI pipeline + DAG	2	✓
MLflow experiment tracking (beyond autolog)	2	✓
Prometheus instrumentation + Grafana	2	✓(30+ metrics)
MLflow APIification (`serve.sh` + `MLproject`)	4	✓
FastAPI endpoints	—	✓(16 endpoints)
Docker Compose (12 services)	—	✓

Table 13: Rubric coverage summary